

Documentação de Arquitetura de Software — Deviante

Modelo arc42 · C4 Model · UML · Entidade-Relacionamento · Software Architecture Canvas

Alander Menezes Arantes de Ávila

Bernardo Creplive Vieira

Emanuelle Skolut Jose

Murilo Regnier Stange

Setembro de 2026

Sobre este documento. Documentação **parcial** da arquitetura do software **Deviante**, seguindo o template oficial **arc42** (12 seções). Os princípios de arquitetura de soluções em nuvem ainda **não** são avaliados por completo nesta entrega. Todos os diagramas exigidos estão embutidos nas seções correspondentes: **C4 Model** (Níveis 1 a 4), **UML** (Sequência, Classes e Componentes no Nível 4), **Diagrama Entidade-Relacionamento** e **Software Architecture Canvas**.

Disciplina: Arquitetura e Soluções Cloud (Turma 6º B) — Engenharia de Software (Noite), 2026 / 2º Semestre.

Professor: Manoel Valerio da Silveira Neto.

Grupo: Alander Menezes Arantes de Ávila · Bernardo Creplive Vieira · Emanuelle Skolut Jose · Murilo Regnier Stange.

Sumário

Introdução e Metas	3
Visão Geral de Requisitos	3
Metas de Qualidade	4
Stakeholders	4
Restrições da Arquitetura	5
Contexto e Escopo	5
Contexto de negócio	6
C4 — Nível 1 · Diagrama de Contexto do Sistema	6
Contexto técnico	8
Estratégia da Solução	9
Software Architecture Canvas	9
Visão de Blocos de Construção	11
Nível 1 — Contêineres (C4 · Nível 2)	11
Nível 2 — Componentes da Aplicação Backend (C4 · Nível 3)	12
Nível 3 — Código (C4 · Nível 4)	13
C4 Nível 4 · UML de Classes — modelo de domínio	14
C4 Nível 4 · UML de Componentes — Aplicação Backend e interfaces	15
Visão de Runtime	16
C4 Nível 4 · UML de Sequência — upload → mapeamento → análise de drift	16
Visão de Implantação	16
Conceitos Transversais	18
Modelo de domínio persistente (Entidade-Relacionamento)	19
Outros conceitos transversais	20
Decisões de Arquitetura	21
Escolha de stack (linguagens e frameworks)	21
Demais decisões estruturais	22
Requisitos de Qualidade	22
Árvore de qualidade (resumo)	22
Cenários de qualidade (mensuráveis, ISO/IEC 25010)	22
Riscos e Dívida Técnica	24
Glossário	25

Introdução e Metas

O **Deviante** é uma plataforma web de **mineração de processos** e **detecção de desvios (drift) de desempenho** para apoio à **manutenção baseada na condição do equipamento de manufatura**. A partir de um **event log** exportado do sistema de origem (formato CSV ou XES), o sistema reconstrói o processo observado, calcula tempos de execução por atividade e aplica o algoritmo **IPDD/ADWIN** sobre séries temporais de desempenho para identificar **quando e onde** o comportamento do processo mudou — antecipando a decisão de manutenção.

O produto materializa a pesquisa acadêmica de IPDD (Sato & Ruschel) e a implementação de ADWIN/predição de falha (Luiz Picolo) em um produto vivo e operável por um gestor de manutenção.

Visão Geral de Requisitos

Requisitos funcionais essenciais (fluxo principal / happy path da v1):

#	Requisito funcional	Objeto de domínio
RF1	Autenticar o gestor (convite, Google/JWT) e compartilhar processos entre usuários autenticados	Manager / User
RF2	Manter Processos de manufatura (criar, listar, editar; excluir só pelo dono)	Process
RF3	Manter um catálogo global de Atividades normalizadas, reutilizável entre processos	Activity
RF4	Importar um Event Log (CSV/XES), fazer o parse e persistir operações, traces, ocorrências e tempos	Event Log
RF5	Mapear cada rótulo bruto (Operation/alias) para uma Atividade normalizada	Operation → Activity
RF6	Gerar o grafo do processo observado a partir do log totalmente mapeado	(visão derivada)
RF7	Executar análise de drift (IPDD/ADWIN) com parâmetros confirmados e persistir o resultado	Analysis
RF8	Investigar o desvio detectado e registrar a decisão de manutenção proativa	Maintenance
RF9	Monitorar a saúde de equipamentos por parâmetros de máquina (ADWIN → RUL/probabilidade de falha)	Monitoring / Equipment

Metas de Qualidade

Prioridade	Meta de qualidade	Cenário
1	Validade científica dos dados temporais	Início, fim e duração reais de cada evento são persistidos sem perda, para que o grafo e a entrada do ADWIN sejam cientificamente válidos (evidência PIBITI/tese).
2	Fidelidade da detecção	A integração preserva a implementação de pesquisa ADWIN/IPDD (envelopa, não reescreve); um dataset controlado estável não acusa drift, um dataset com drift intencional acusa.
3	Integridade do mapeamento	Nenhuma Atividade é criada silenciosamente; lotes de mapeamento incompletos ou entre processos diferentes são rejeitados.
4	Segurança e isolamento	Acesso apenas por convite; todo endpoint de negócio valida JWT; RLS habilitado nas tabelas de dados.
5	Usabilidade (manipulação direta)	Criação/edição de objetos "no lugar" (estilo Figma), evitando telas-formulário desnecessárias.

Stakeholders

Papel	Quem / contato	Expectativa
Owner / Gestor de Manutenção	Alander Ávila (design@alander.io)	Constrói, publica e decide; roda o produto de ponta a ponta em produção.
Orientador / Mentor	Eduardo Loures (PUCPR, Mecatrônica)	Orienta a pesquisa PIBITI, testa e valida os resultados.
Pesquisador / Mentor	Luiz Pico (mestrando, Bosch)	Fornecer e validar o algoritmo ADWIN/predição; testa.

Restrições da Arquitetura

O que o projeto é **obrigado a respeitar** (regra, não escolha). O que “pode dar errado” é **risco** e vive na Seção 11.

Restrições técnicas

- **Fronteira de responsabilidade fixa:** o **Kotlin/Ktor** é dono de **toda** a persistência e das regras de negócio; o serviço **Python/FastAPI** é **computação sem estado** (não abre conexão com o banco). A web (React) só consome a Aplicação Backend — nunca fala com o Postgres diretamente nem reimplementa regra de negócio.
- **Mineração de processo em Python:** PM4Py/river não têm equivalente prático na JVM; por isso o parsing de event log (mineração) e a detecção de desvios ficam num serviço Python separado, chamado pela Aplicação Backend.
- **Envelopar, não reescrever:** o código de pesquisa de ADWIN/predição (Adaptive-Detection-of-Performance-Related-Temporal-Drifts) deve ser **adaptado/importado**, preservando a proveniência científica — não reimplementado como código de produto.
- **Plataforma gerenciada:** identidade e banco vêm do **Supabase** (Auth + Postgres); autenticação **por convite** (sem auto-cadastro).

Restrições organizacionais / convenções

- **v1 é somente fluxo principal (PIBITI/tese):** apenas o happy path de cada caso de uso; fluxos alternativos/erro ficam fora desta versão.
- **Idioma:** a **UI é PT-BR**; **código e documentação de conhecimento são em inglês**. Este documento acadêmico é redigido em PT-BR.
- **SQL versionado** em `data/schema/deviante/`; sempre `schema.tabela`; sem Flyway/automigração dentro da Aplicação Backend.

Restrições legais / de dados

- **Proteção de dados (LGPD):** o produto trata dados pessoais (e-mail e identidade dos usuários) e dados operacionais de empresas; o tratamento deve cumprir a **LGPD** — base legal, minimização, isolamento por titular e possibilidade de exclusão. Acesso apenas por convite; sem compartilhamento com terceiros.
- **Provedor único:** identidade e banco vivem no **mesmo projeto Supabase**; a v1 não assume múltiplos provedores de banco.

Contexto e Escopo

Delimita a fronteira do Deviante em relação aos seus usuários e aos sistemas externos. Corresponde ao **C4 Model — Nível 1 (Contexto do Sistema)**.

- **Dentro do escopo:** o **Deviante** — Web App, Aplicação Backend, Serviço de Mineração e Banco de Dados. O banco e o serviço de mineração são **internos** ao sistema (não são sistemas externos).
- **Fora do escopo (atores e sistemas vizinhos):** o **Gestor de Manutenção** (usuário) e a **Autenticação Supabase** «externo» (provedor de identidade). O event log já existe (exportado do chão de fábrica) e é apenas importado.

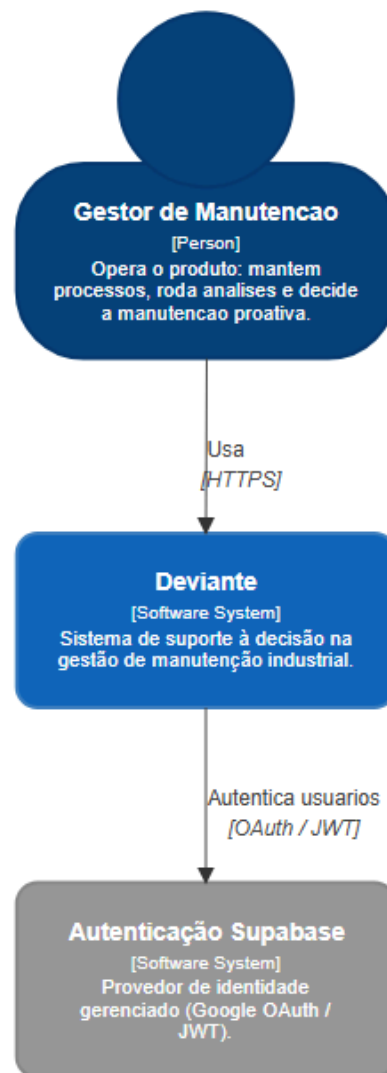
Contexto de negócio

Ator: o **Gestor de Manutenção** (usuário autenticado que opera o produto), que já possui o event log e o importa para o Deviante. O único sistema **externo** é a **Autenticação Supabase** (provedor de identidade); o **Banco de Dados** e o **Serviço de Mineração** pertencem ao próprio Deviante, não são externos. A decomposição em contêineres vem no Nível 2.

C4 — Nível 1 · Diagrama de Contexto do Sistema

Um único ator opera o Deviante e um único sistema externo o apoia: o **Gestor de Manutenção** usa o produto, e a **Autenticação Supabase** provê a identidade. Tudo o mais é interno.

C4 Nivel 1 - Contexto do Sistema: Deviante



Contexto técnico

Interface	Canal / protocolo	Dados
Navegador → Web App	HTTPS	Interação do usuário (SPA)
Web App → Autenticação Supabase	supabase-js (OAuth/JWT)	Login, sessão, token
Web App → Aplicação Backend	REST /api/* (JSON sobre HTTPS)	Dados de negócio
Aplicação Backend → Banco de Dados	JDBC / HikariCP	Leitura e escrita persistente
Aplicação Backend → Serviço de Mineração	REST (/parse, /detect)	Event log e séries temporais
Gestor → Web App	upload de arquivo	Event log CSV/XES

Estratégia da Solução

Linha mestra (uma página, sem detalhar módulos nem servidores): um front web (React) consome uma **Aplicação Backend** (Kotlin/Ktor) que concentra regras de negócio e persistência; a **mineração de processo** e a **detecção de desvios** ficam num **serviço Python (FastAPI) sem estado**, chamado pela Aplicação Backend; o **Postgres** é a única fonte de verdade. Onde isso roda (nuvem, ambientes) é **Implantação (\$7)**; a fronteira com atores/externos é **Contexto (\$3)** — não aqui.

Decisões estruturantes que endereçam as metas de qualidade:

Meta de qualidade	Abordagem de solução
Validade dos dados temporais	O parser do serviço Python devolve início/fim/duração por evento e por trace; o Kotlin persiste tudo em <code>traces / trace_events</code> (colunas de tempo dedicadas) antes de qualquer análise.
Fidelidade da detecção	O serviço FastAPI envelopa os scripts de pesquisa de ADWIN/IPDD como um adaptador fino (<code>/detect</code>), preservando o algoritmo original.
Separação de responsabilidades	Kotlin = regras + persistência; Python = computação sem estado; React = apresentação.
Integridade / segurança	Auth por convite (Supabase), validação de JWT via JWKS em cada rota, RLS no Postgres, mapeamento validado em lote.
Manutenibilidade	Padrão <code>*Table + *Repository</code> (Exposed/JDBC) por agregado; SQL versionado; um único ponto de verdade para os objetos (OOUX).

Software Architecture Canvas

Visão de uma página da arquitetura (modelo *Architecture Communication Canvas*, dos autores do arc42).

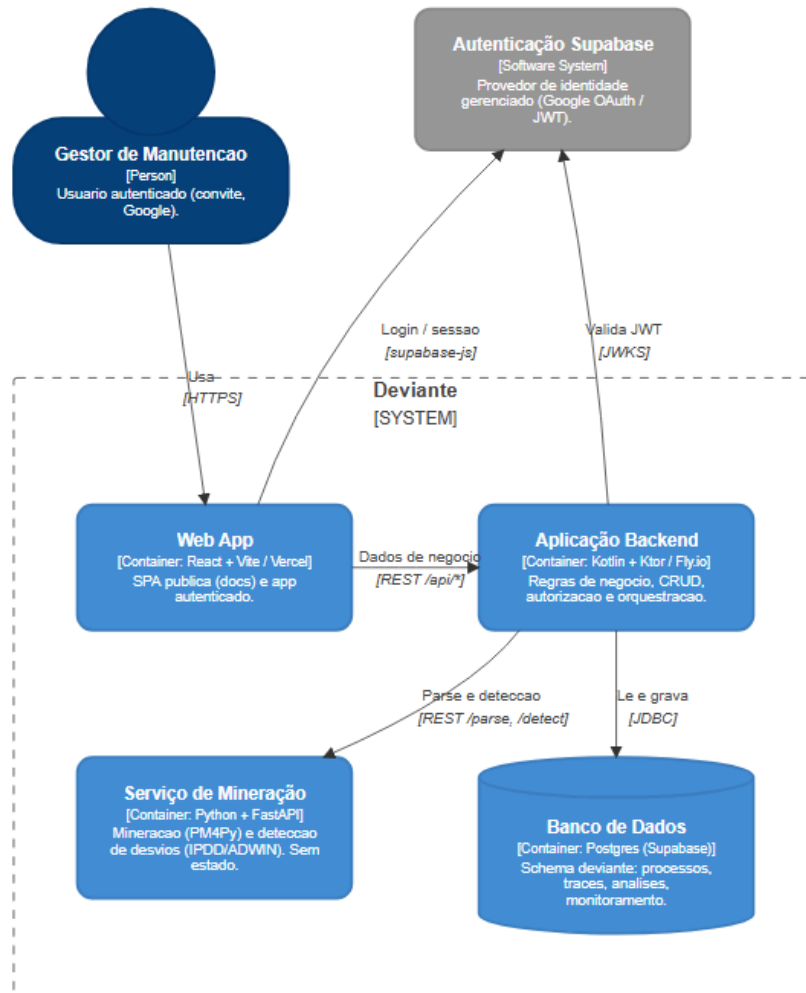
Bloco	Conteúdo
Proposta de valor	Antecipar a manutenção detectando <i>drift</i> de desempenho no event log, transformando a pesquisa IPDD/ADWIN em um produto operável por um gestor.
Stakeholders	Gestor de Manutenção (owner); mentores/testadores (Eduardo Loures, Luiz Pícolo); autoria científica IPDD (Denise Sato, Edson Ruschel).
Requisitos de qualidade (Top)	1. Validade científica dos tempos; 2. Fidelidade da detecção (ADWIN); 3. Integridade do mapeamento; 4. Segurança (convite, JWT, RLS).
Contexto de negócio	Ator único: Gestor de Manutenção, que já possui o event log. Sistema externo: Autenticação Supabase (identidade).
Contexto técnico	Web (HTTPS) → Aplicação Backend (REST /api/*) → Banco (JDBC); Aplicação Backend → Serviço de Mineração (/parse, /detect); Autenticação Supabase (JWKS).
Estratégia de solução	Três contêineres + banco: React/Vite (Vercel), Kotlin/Ktor (Fly.io), Python/FastAPI, Postgres (Supabase). Kotlin persiste; Python só computa.
Blocos de construção	Aplicação Backend: Rotas/Auth, Serviços de Domínio (Process, Activity, Mapping, Analysis, Monitoring), Cliente de Mineração, Repositories.
Tecnologias	Kotlin, Ktor, Exposed, HikariCP; Python, FastAPI, PM4Py, river/ADWIN; React, Vite, Tailwind; Supabase (Auth + Postgres).
Riscos & pendências	RLS a completar; contrato de layout do grafo; visualização 3D de máquina (gITF) em aberto; modelo de auth ainda simples (owner/mentor).

Visão de Blocos de Construção

Decomposição estática hierárquica, na mesma lógica de *zoom* do C4: o Nível 1 mostra os contêineres; o Nível 2 abre a Aplicação Backend em componentes; o Nível 3 (código) detalha o modelo de classes e os componentes internos.

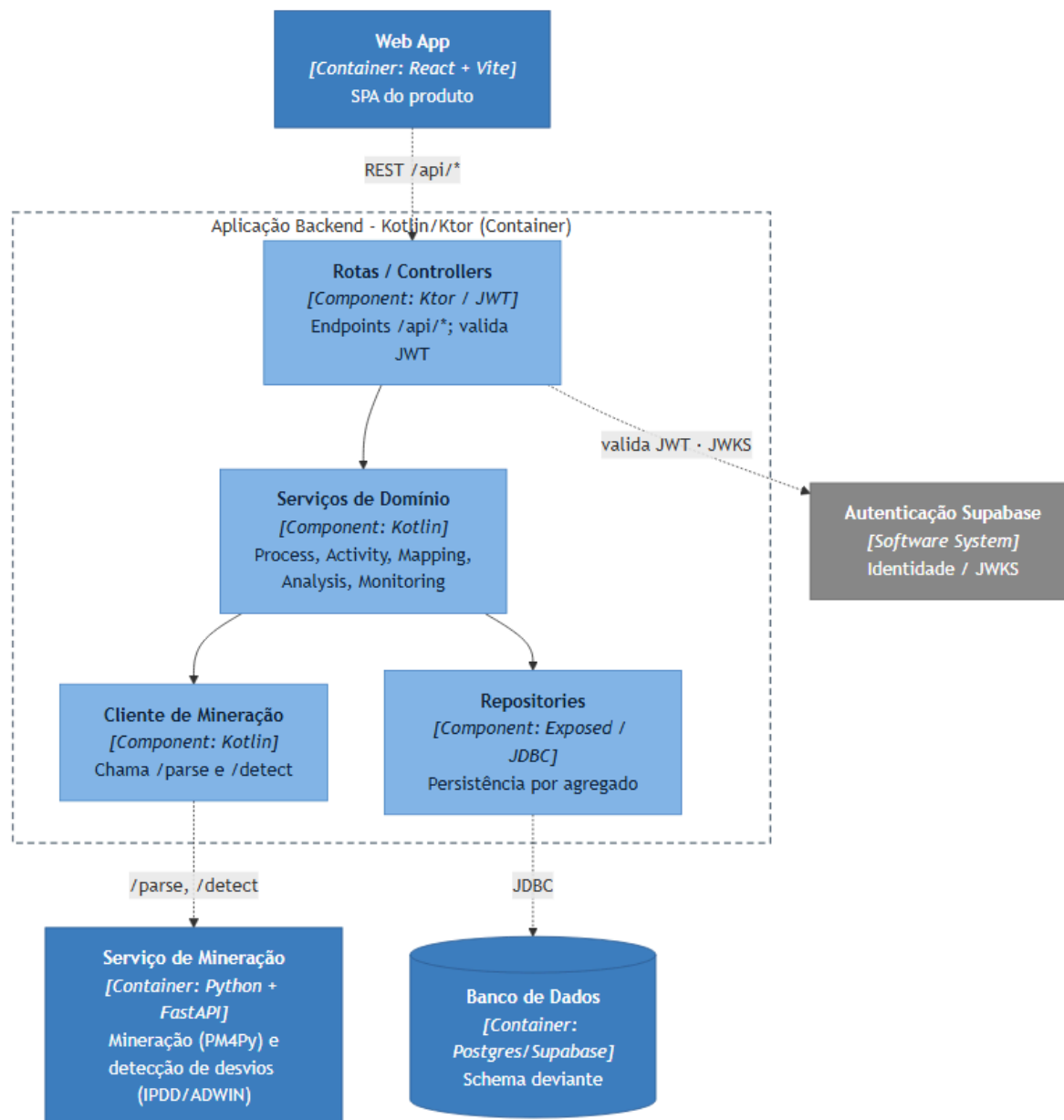
Nível 1 — Contêineres (C4 · Nível 2)

Três contêineres implantáveis do Deviante mais o seu banco de dados; a Autenticação Supabase aparece como serviço externo de identidade.



Nível 2 — Componentes da Aplicação Backend (C4 · Nível 3)

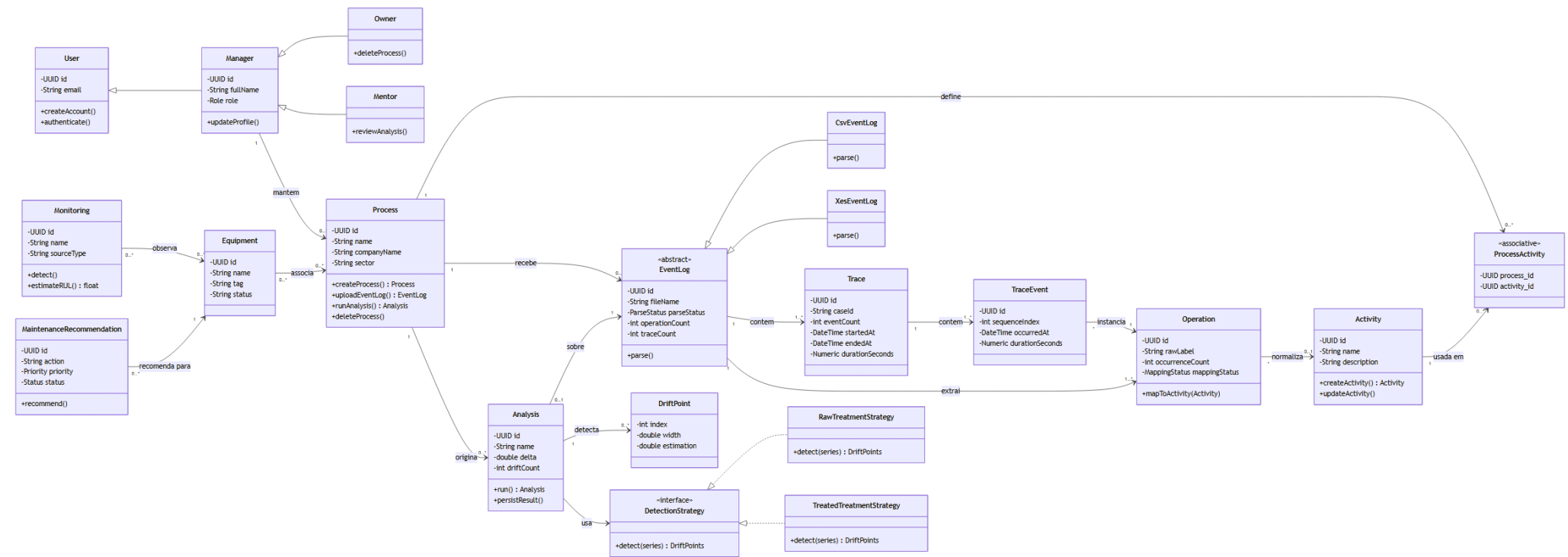
Os **componentes de domínio** espelham os objetos do OOIX (Process, Activity, Mapping, Analysis, Monitoring); os **componentes técnicos** (Rotas/Auth, Cliente de Mineração, Repositories) não têm objeto de domínio correspondente. Este nível usa a notação C4 desenhada como *flowchart* estilizado para leitura limpa (fronteira tracejada, tags [Container]/[Component], externos em cinza).



Nível 3 — Código (C4 · Nível 4)

O nível de código do C4 é detalhado por três diagramas **UML**: **Classes** (modelo de domínio), **Componentes** (componentes e interfaces da Aplicação Backend) e **Sequência** (na Seção 6 · Runtime).

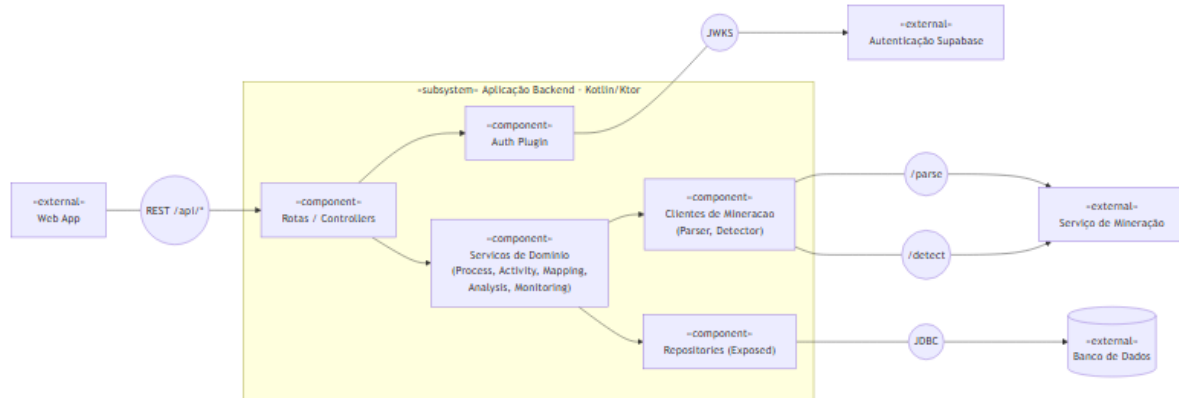
C4 Nível 4 · UML de Classes — modelo de domínio



Notas de modelagem. As especializações Owner / Mentor e CsvEventLog / XesEventLog são **conceituais**: em runtime correspondem, respectivamente, ao campo `role` de `Manager` e ao campo `format` de `EventLog`. `ProcessActivity` é a **classe associativa** que materializa o modelo definido (junção `process_activities`). `DetectionStrategy` reflete o parâmetro real `treatment` (`raw` / `treated`) do serviço de mineração — o padrão **Strategy** seleciona qual pipeline de pesquisa reproduzir.

C4 Nível 4 · UML de Componentes — Aplicação Backend e interfaces

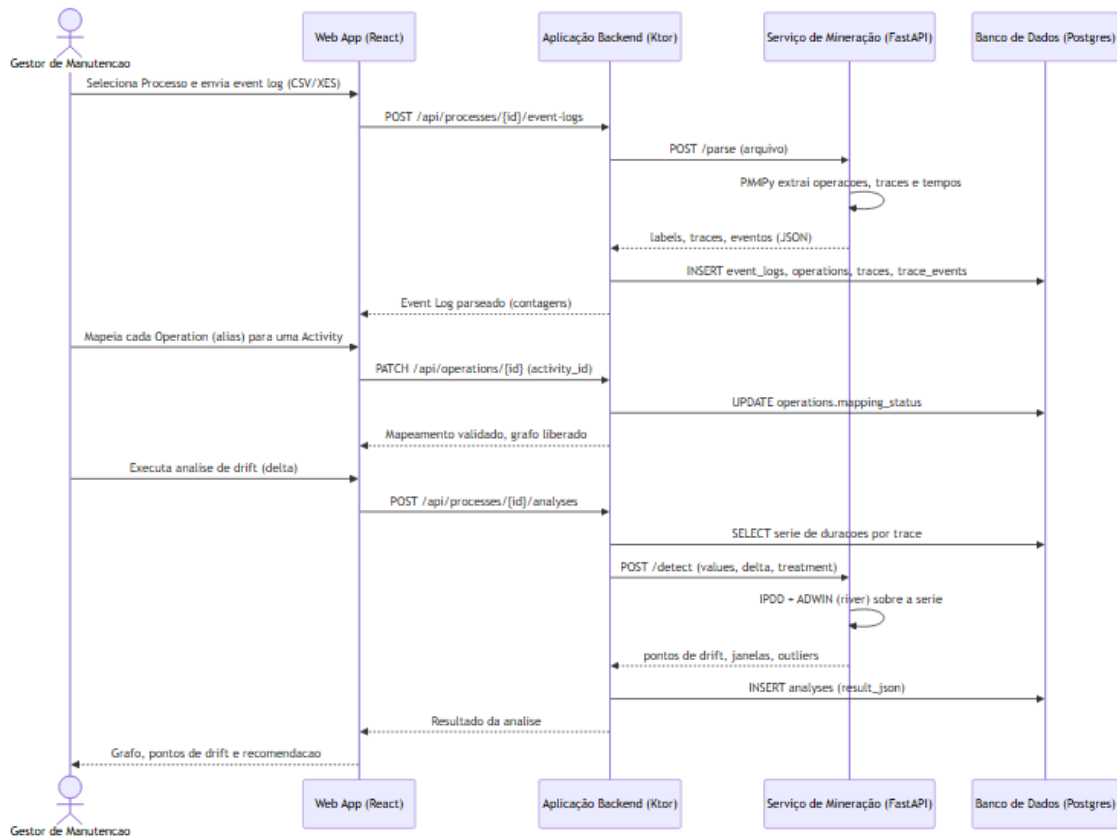
Componentes («component») da Aplicação Backend com suas **interfaces providas e requeridas** (notação de lollipop). Os atores externos (Web, Mineração, Auth, Banco) aparecem nas bordas.



Visão de Runtime

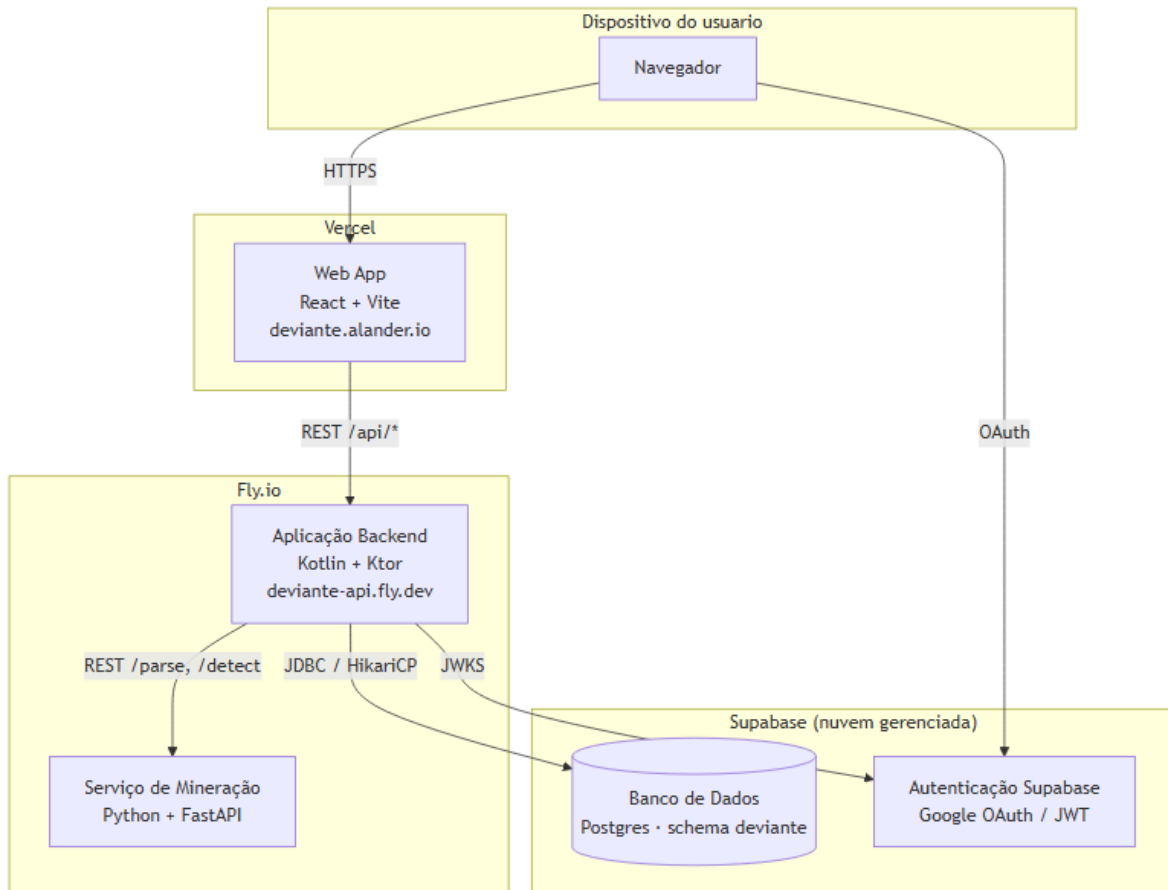
Cenário dinâmico central: **importar o event log, mapear e rodar a análise de drift**. É a sequência que exercita os três contêineres e ambos os endpoints do serviço de mineração (/parse para a mineração com PM4Py, /detect para a detecção de desvios com IPDD/ADWIN).

C4 Nível 4 · UML de Sequência — upload → mapeamento → análise de drift



Visão de Implantação

Mapeamento dos contêineres para a infraestrutura de execução e os canais entre os nós.



Nota (entrega parcial): os detalhes de arquitetura de nuvem — escalabilidade, observabilidade, custo e topologia de rede — serão aprofundados na entrega completa. Aqui está a topologia de implantação atual verificada.

Conceitos Transversais

Padrões recorrentes que atravessam os blocos de construção.

Persistência no Postgres (schema `deviante`). O ER abaixo reflete o **banco real em produção** (projeto Supabase `ydjtrcjxhtmygtmebrk`). Núcleo de mineração de processo à esquerda; cluster de monitoramento/manutenção à direita.

Modelo de domínio persistente (Entidade-Relacionamento)

Outros conceitos transversais

- **Segurança / autenticação (política global):** identidade na Autenticação Supabase (Google OAuth, por convite); **todo** endpoint de negócio valida o JWT (JWKS) antes de executar; RLS habilitado nas tabelas do schema `deviante`. Papéis de runtime: `owner` (exclui processos) e `mentor` (consulta/edita, não exclui).
- **Logging / observabilidade (política global):** requisições de negócio e falhas são registradas em um formato único que atravessa os três contêineres; a análise guarda sua proveniência (método, delta, tratamento) em `analyses.result_json` para auditoria.
- **Processo definido × processo observado:** `process_activities` registra o **modelo definido** (atividades escolhidas para o processo); o grafo `/graph` é o **processo observado**, derivado só dos eventos mapeados do último log válido. Uma atividade sem ocorrência no log não recebe frequência/duração inferidas.
- **Dados temporais:** início, fim e duração de cada evento/trace são persistidos (`trace_events.duration_seconds`, `traces.*`) — pré-condição para a validade científica do ADWIN.
- **Mineração × detecção:** a mineração de processo (PM4Py, endpoint `/parse`) reconstrói operações, traces e tempos; a detecção de desvios (IPDD/ADWIN, endpoint `/detect`) roda sobre as séries temporais. São responsabilidades distintas do mesmo serviço de mineração.
- **Proveniência de pesquisa:** a computação de drift é um adaptador fino sobre os scripts de ADWIN/IPDD; o `result_json` da análise guarda a proveniência (método, delta, tratamento) junto ao resultado.
- **Tratamento de erros (política global):** erros de validação retornam HTTP 4xx com mensagem em **PT-BR**; falhas do serviço de mineração são propagadas com contexto pela Aplicação Backend, nunca silenciadas. Exemplo: o parse valida tamanho/vazio e o `event_logs.parse_status` reflete o estado (`pending` / `parsing` / `parsed` / `failed`).
- **Internacionalização (i18n):** a UI é **PT-BR**; identificadores de código, logs e o vocabulário técnico ficam em inglês. Mensagens ao usuário são sempre em PT-BR.

Decisões de Arquitetura

Registro de decisões (ADR) mais relevantes.

Cada decisão traz a **alternativa considerada**, a **justificativa** e as **consequências** (formato ADR).

Escolha de stack (linguagens e frameworks)

ADR	Decisão	Alternativa considerada	Justificativa	Consequência
ADR-1	React (Vite) no front web	Angular/Vue, ou UI fora do padrão do design	Alta aderência ao design feito no Figma (onde as interfaces são prototipadas e implementadas) e caminho para reaproveitar React (React Native) nas futuras versões Android/iOS	Ecossistema JS/SPA; componentes reaproveitáveis no mobile futuro.
ADR-2	Kotlin/Ktor no backend	Node/TypeScript ou Python como linguagem do backend	OOP nativa (sustenta o modelo de domínio rico da §5) e afinidade/performance com Android (SO mais popular no Brasil), preparando as futuras versões mobile	JVM; linguagem alinhada ao roadmap mobile Android.

Demais decisões estruturais

ADR	Decisão	Alternativa considerada	Justificativa	Consequência
ADR-3	Serviço Python separado para a mineração	Reimplementar PM4Py/ADWIN na JVM	Não há equivalente prático na JVM; reescrever seria caro e arriscado	Uma fronteira de rede a mais; a Aplicação Backend orquestra e persiste.
ADR-4	Kotlin dono da persistência ; Python sem estado	A Web falar direto com o Postgres (<code>supabase-js</code>)	Evitar duas fontes de verdade e regra de negócio duplicada	Regras/consistência num só lugar; Python é computação pura.
ADR-5	Envelopar, não reescrever o ADWIN de pesquisa	Reescrever o algoritmo como código de produto	Preservar a proveniência científica (tese/PIBITI)	Adaptador fino <code>/detect</code> ; scripts originais quase intocados.
ADR-6	Chamada síncrona ao serviço de mineração	Fila/assíncrono (ex.: SQS) entre backend e mineração	Análise é sob demanda e rápida; desacoplar por fila é desnecessário na v1	Fluxo simples; sob carga futura pode virar fila (ver §11).
ADR-7	Supabase (Auth + Postgres), por convite	Infra própria (auth + Postgres self-hosted)	Time pequeno, foco no produto, sem auto-cadastro na v1	Menos código de infra; lock-in do provedor (ver §11).
ADR-8	Resultado de drift em <code>analyses.result.json</code>	Tabela <code>drift_results</code> normalizada	O resultado é um documento por execução; leitura/reabertura simples	Menos tabelas; consultas por documento.

Requisitos de Qualidade

Refina as metas da Seção 1 em cenários verificáveis.

Árvore de qualidade (resumo)

- **Confiabilidade / Validade científica** → precisão temporal, fidelidade de detecção.
- **Segurança** → autenticação, autorização, isolamento (RLS).
- **Manutenibilidade** → separação de responsabilidades, padrões de persistência.
- **Usabilidade** → manipulação direta, consistência visual.

Cenários de qualidade (mensuráveis, ISO/IEC 25010)

Metas do que será **avaliado** (não substituem teste).

ID	Característica (ISO 25010)	Cenário e meta mensurável
Q1	Eficiência de desempenho	O parse de <code>Prod1Torno.csv</code> (13.053 eventos, 3.282 traces) conclui e persiste em < 30 s .
Q2	Eficiência de desempenho	A análise de drift (<code>/detect</code>) sobre ~3.300 traces retorna em < 5 s em ≥ 95% das execuções.
Q3	Correção funcional / Confiabilidade	<code>ST_01.xes</code> (controle) não acusa drift; <code>DR_01.xes</code> identifica o ponto — resultado reprodutível a cada execução com o mesmo <code>delta</code> .
Q4	Segurança	100% dos endpoints de negócio exigem JWT válido (401 sem token); um <code>mentor</code> não exclui Processo; RLS isola dados por usuário.
Q5	Integridade	Lote de mapeamento incompleto ou entre processos diferentes é rejeitado ; 0 Atividades criadas silenciosamente.
Q6	Confiabilidade (persistência)	Após recarregar a página, 100% dos mapeamentos e o resultado da análise persistem inalterados.
Q7	Usabilidade	Criação/edição de objetos acontece no lugar (estilo Figma), sem tela-formulário separada; o grafo só é liberado após o log estar totalmente mapeado.

Riscos e Dívida Técnica

Incerteza (**risco**) e passivo (**dívida técnica**) — o que pode doer depois. Regra obrigatória é **restrição** (§2), não risco. Ordenados por prioridade.

Item	Tipo	Impacto	Mitigação
Lock-in de fornecedor (Supabase, Fly.io, Vercel)	Risco	Migração custosa; indisponibilidade do provedor derruba o produto	Isolar o acesso atrás da Aplicação Backend; manter o SQL versionado para portar o schema.
Dependência do código de pesquisa (ADWIN/PM4Py de terceiro)	Risco	Bug ou mudança no upstream afeta a análise	Envelopar (não forkar), fixar versões, guardar a proveniência no <code>result_json</code> .
RLS incompleto em tabelas novas	Dívida	Vazamento/edição indevida entre usuários	Completar políticas por UC antes da avaliação completa de nuvem.
Sem cache de análise/grafos	Dívida	Cada análise recomputa; pode ficar lento sob carga	Introduzir cache/persistência de resultado; hoje o <code>result_json</code> já guarda o último.
Autorização historicamente por criador em algumas rotas	Dívida	Listagem/edição inconsistente de processos compartilhados	Corrigir a autorização como uma única mudança (UC2), não rota a rota.
Modelo de auth simples (owner/mentor)	Dívida	Não cobre papéis futuros	Evoluir o modelo de papéis quando novos casos surgirem.
Visualização 3D de máquina (gITF) em aberto	Risco	Feature de equipamento incompleta	Decidir formato/armazenamento; ausência de modelo é estado válido.

Glossário

Termo	Definição
IPDD	Componente de detecção de desvios (drift) do Serviço de Mineração (FastAPI), baseado no framework <i>Interactive Process Drift Detection</i> (Sato & Ruschel).
ADWIN	<i>Adaptive Windowing</i> — algoritmo de detecção de mudança em fluxos de dados (via <code>river</code>).
PM4Py	Biblioteca Python de <i>process mining</i> usada na mineração (endpoint <code>/parse</code>).
Drift (de desempenho)	Mudança estatisticamente significativa no comportamento temporal do processo (ex.: aumento do tempo de execução).
Event log	Registro de eventos do processo (CSV/XES) com caso, atividade e carimbo de tempo.
Trace / Case	Uma instância/execução do processo (sequência ordenada de eventos).
Operation (alias)	Rótulo bruto do event log, variante de nome de uma atividade.
Activity	Conceito lógico normalizado para o qual vários rótulos (Operations) convergem.
Sojourn time	Tempo de permanência de um evento/atividade (duração).
Analysis (Análise)	Uma execução de detecção de drift (IPDD/ADWIN) sobre um Event Log, com seus parâmetros e resultado (<code>analyses.result_json</code>).
Monitoring (Monitoramento)	Acompanhamento da saúde de um Equipamento por parâmetros de máquina, com detecção via ADWIN e estimativa de RUL.
JWT / JWKS	Token de autenticação (JWT) emitido pelo provedor de identidade; validado pela API contra as chaves públicas (JWKS).
LGPD	Lei Geral de Proteção de Dados (Brasil) — obrigação legal sobre o tratamento de dados pessoais.
Processo definido × observado	Definido = atividades escolhidas (<code>process_activities</code>); observado = grafo derivado dos eventos mapeados.
RUL	<i>Remaining Useful Life</i> — vida útil remanescente estimada de um equipamento.
RLS	<i>Row-Level Security</i> do Postgres.
C4 Model	Modelo de visualização de arquitetura em 4 níveis: Contexto, Contêiner, Componente e Código.
arc42	Template de documentação de arquitetura de software (12 seções).